Programmbeispiele (Python) - Raspberry Pi, Kamera u. Pixhawk

Raspberry Pi 5 (Companion Computer)

Camera Modul 3

Raspberry Pi

pixhawk

Pixhawk 6C

Inhalt

1. Python-Module 2
   a. Picamera2 2
   b. OpenCV 2
   c. MAVSDK für Python 2
2. Raspberry Pi Kamera-Ansteuerung 3
   a. Vorschau mit QT und Laden von Kamera-Konfigurationen (Tuning-File) 3
   b. Bildsequenzaufnahme 4
   c. Videoaufnahme 5
   d. Objekterkennung (Farben) mit OpenCV 6
3. Datenaustausch zwischen Raspberry Pi und Pixhawk 9
   a. Abruf von Telemetrie-Parametern 9
   b. Vorgabe von Wegpunkten (bzw. Positionen im Offboard-Modus) 9
   c. Synchronisieren von Bildern mit Telemetrie-Daten und speichern in CSV-Datei 11

1. Python-Module

a. Picamera2 picamera2

- picamera2 ist eine Python-Bibliothek, um den Zugriff auf die Raspberry Pi Kamera zu ermöglichen
- picamera2 basiert auf der libcamera-Bibliothek und steuert Kamerafunktionen auf einem Raspberry Pi, wie z.B. das Aufnehmen von Bildern und Videos
- picamera2 ist auf GitHub verfügbar: https://github.com/raspberrypi/picamera2
- Dokumentation https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf und weitere Beispiele auf der GitHub-Repository

b. OpenCV

- OpenCV (Open Source Computer Vision Library) ist Bibliothek für Bild- und Videobearbeitung (z.B. Objekterkennung)
- Dokumentation und Beispiele sind auf der offiziellen OpenCV-Webseite: https://opencv.org/ sowie auf GitHub verfügbar.
- OpenCV kann über den Python-Pakete-Manager „pip“ installiert werden

c. MAVSDK für Python MAVSDK

- MAVSDK f. Python ist eine Python-Bibliothek zur Kommunikation mit Drohnen, welche das MAVLink-Protokoll verwenden
- MAVSDK kann verwendet werden, um Drohnen zu steuern und Telemetrie-Daten über das MAVLink-Protokoll auszutauschen.
- Die offizielle Dokumentation ist auf der MAVSDK-Webseite: https://mavsdk.mavlink.io/main/en/python/ sowie auf GitHub https://github.com/mavlink/MAVSDK-Python/tree/main verfügbar.

Picamera2 - Bibliothek (https://github.com/raspberrypi/picamera2):

2. Raspberry Pi Kamera-Ansteuerung

a. Vorschau mit QT und Laden von Kamera-Konfigurationen (Tuning-File)

```
#!/usr/bin/python3

# picamera2-library Example of setting controls using Tuning-File and the
"direct" attribute method.

'''
Beschreibung: Programm für Raspberry Pi Camera Vorschau mit Tuning-File
'''

from picamera2 import Picamera2, Preview
from picamera2.controls import Controls
import time

# Laden des Tuning-Files
tuning = Picamera2.load_tuning_file("imx708.json") # Für Raspberry Pi
Kamera IR "imx708_noir.json" --> Tuning-Files in
/usr/share/libcamera/ipa/rpi/pisp

# Erstellen und Konfigurieren der Kamera mit dem geladenen Tuning-File
picam2 = Picamera2(tuning=tuning)
picam2.configure(picam2.create_preview_configuration())

# Starten d. Kameravorschau
#picam2.start_preview(Preview.QTGL) # Bei Remote-Desktop Verbindung QTGL
nicht möglich, bei VNC-Verbindung schon
picam2.start_preview(Preview.QT)

# Starten d. Kamera
picam2.start()

# Manuelle Einstellung einzelner Kamera-Parameter
with picam2.controls as ctrl:
#     Automatische Belichtung deaktivieren
      ctrl.AeEnable = False
#     Automatischen Weißabgleich deaktivieren
      ctrl.AwbEnable = False
#     Analoger Gain (Helligkeit ohne Rauschen) einstellen
      ctrl.AnalogueGain = 5.0
#     Belichtungszeit in Mikrosekunden (500 bedeutet 0.5ms) einstellen
      ctrl.ExposureTime = 3000
#     Rauschreduktion aktivieren (Modus 4)
      ctrl.NoiseReductionMode = 4
#     Bildschärfe einstellen
      ctrl.Sharpness = 6

# Kamera ist mit den festgelegten Einstellungen und dem Tuning-File
konfiguriert
time.sleep(2)
print("Bildaufnahme gestartet")

time.sleep(30)

# Kamera stoppen
picam2.stop()
```

b. Bildsequenzaufnahme

```
#!/usr/bin/python3

'''
Beschreibung: Programm zur Aufnahme und Speichern von Bildsequenzen +
Laden von Kamera-Einstellungen mit einem Tuning-File
Variable Parameter: Dateispeicherort (Zeile 47), Anzahl d. Bilder (Zeile
51) und Zeitintervall zwischen den Bildaufnahmen (Zeile 53)
'''

from picamera2 import Picamera2, Preview
from picamera2.controls import Controls
import time
import os

# Laden des Tuning-Files "imx708.json"
tuning = Picamera2.load_tuning_file("imx708.json") # Für Raspberry Pi
Kamera IR "imx708_noir.json" --> Tuning-Files in
/usr/share/libcamera/ipa/rpi/pisp

# Kamera initialisieren und konfigurieren mit dem geladenen Tuning-File
picam2 = Picamera2(tuning=tuning)
picam2.configure(picam2.create_preview_configuration())

# Starten d. Kameravorschau
# picam2.start_preview(Preview.QTGL) # Bei Remote-Desktop Verbindung QTGL
nicht möglich, nur via VNC-Verbindung QTGL möglich
picam2.start_preview(Preview.QT)

# Starten d. Kamera
picam2.start()

# Manuelle Einstellung einzelner Kamera-Parameter
with picam2.controls as ctrl:
#    Automatische Belichtung deaktivieren
     ctrl.AeEnable = False
#    Automatischen Weißabgleich deaktivieren
     ctrl.AwbEnable = False
#    Analoger Gain (Helligkeit ohne Rauschen) einstellen
     ctrl.AnalogueGain = 2.0
#    Belichtungszeit in Mikrosekunden (500 bedeutet 0.5ms) einstellen
     ctrl.ExposureTime = 3000
#    Rauschreduktion aktivieren (Modus 4)
     ctrl.NoiseReductionMode = 4
#    Bildschärfe einstellen
     ctrl.Sharpness = 6

# Kamera ist mit den festgelegten Einstellungen und dem Tuning-File
konfiguriert
time.sleep(2)
print("Bildaufnahme gestartet")

# Speicherverzeichnis erstellen (falls nicht vorhanden)
SAVE_PATH = '/home/aviator/Desktop/Bildsequenz'
os.makedirs(SAVE_PATH, exist_ok=True)

# Bilder in Schleife als .jpg speichern
for i in range(10):        # Anzahl der Bilder, die aufgenommen werden
soll (variabel)
```

```
    picam2.capture_file(f'{SAVE_PATH}/image_{i}.jpg') # Speichern der
Bilder mit fortlaufendem Index als image_i.jpg
    time.sleep(1)          # Zeitintervall zwischen den Bildern, 1
Sekunde (variabel)

# Kamera stoppen
picam2.stop()
```

c. Videoaufnahme

```
'''
Beschreibung: Programm zur Aufnahme eines Videos im mp4-Format + Laden
von Kamera-Einstellungen mit einem Tuning-File
Variable Parameter: Dateispeicherort (Zeile 45 und Videolänge (Zeile 52)
'''

#!/usr/bin/python3
from picamera2 import Picamera2, Preview
from picamera2.controls import Controls
from picamera2.encoders import H264Encoder
from picamera2.outputs import FfmpegOutput
import time
import os

# Laden eines Tuning-Files
tuning = Picamera2.load_tuning_file("imx708.json") # Für Raspberry Pi
Kamera IR "imx708_noir.json" --> Tuning-Files in
/usr/share/libcamera/ipa/rpi/pisp

# Erstellen und Konfigurieren der Kamera mit dem geladenen Tuning-File
picam2 = Picamera2(tuning=tuning)
picam2.configure(picam2.create_preview_configuration())

# Starten d. Kameravorschau
# picam2.start_preview(Preview.QTGL) # Bei Remote-Desktop Verbindung QTGL
nicht möglich, bei VNC-Verbindung schon
picam2.start_preview(Preview.QT)

# Manuelle Einstellung einzelner Kamera-Parameter
with picam2.controls as ctrl:
#     Automatische Belichtung deaktivieren
      ctrl.AeEnable = False
#     Automatischen Weißabgleich deaktivieren
      ctrl.AwbEnable = False
#     Analoger Gain (Helligkeit ohne Rauschen) einstellen
      ctrl.AnalogueGain = 6.0
#     Belichtungszeit in Mikrosekunden (500 bedeutet 0.5ms) einstellen
      ctrl.ExposureTime = 3000
#     Rauschreduktion aktivieren (Modus 4)
      ctrl.NoiseReductionMode = 4
#     Bildschärfe einstellen
      ctrl.Sharpness = 6

# Kamera ist mit den festgelegten Einstellungen und dem Tuning-File
konfiguriert
time.sleep(2)
print("Bildaufnahme starten")

# Speicherverzeichnis erstellen (falls nicht vorhanden)
SAVE_PATH = '/home/aviator/Desktop/Video'
```

```
os.makedirs(SAVE_PATH, exist_ok=True)

encoder = H264Encoder(10000000)
output = FfmpegOutput(f'{SAVE_PATH}/test.mp4')

picam2.start_recording(encoder, output)
time.sleep(10)
picam2.stop_recording()
```

d. Objekterkennung (Farben) mit OpenCV

```
'''
Beschreibung: Programm zum Zählen von erkannten Farbobjekten Blau, Rot
und Gelb
Anpassbare Parameter: Hinzufügen von Farbprofilen
'''

import cv2
from picamera2 import Picamera2

# Definieren von Farbprofilen für die Farberkennung
def colorProfiles(n):
    if n == 0:
        name = "Blau"
        hsv_lower = (95,100,100)
        hsv_upper = (115,255,255)
        return (name,hsv_lower,hsv_upper)
    if n == 1:
        name = "Rot"
        hsv_lower = (160,100,100)
        hsv_upper = (180,255,255)
        return (name,hsv_lower,hsv_upper)
    if n == 2:
        name = "Gelb"
        hsv_lower = (15, 100, 100)
        hsv_upper = (35, 255, 255)
        return (name,hsv_lower,hsv_upper)
    if n == 3:
        name = "Rot2"
        hsv_lower = (0,100,100)
        hsv_upper = (12,255,255)
        return (name,hsv_lower,hsv_upper)

# Mindestfläche des zu erkennden Objekts (z.B. 3000 Pixel)
min_area = 3000                                          # mögliche
Verbesserung durch dynamische Anpassung in Abhängigkeit von der Flughöhe

counter = 0

blueObject = False
redObject = False
yellowObject = False
redObject2 = False

# Starten eines neuen Threads für ein OpenCV-Fenster
cv2.startWindowThread()

#picam2 = Picamera2()
tuning = Picamera2.load_tuning_file("imx708.json")       # Für Raspberry
Pi IR Kamera "imx708_noir.json"
```

```
picam2 = Picamera2(tuning=tuning)

picam2.configure(picam2.create_preview_configuration(main={"format":
'XRGB8888', "size": (640, 480)}))
#
# picam2.configure(picam2.create_preview_configuration(main={"format":
'XRGB8888', "size": (1280, 720)}))

picam2.start()

with picam2.controls as ctrl:
    ctrl.AeEnable = False        # Automatische Belichtung deaktivieren
    ctrl.AwbEnable = False       # Automatischen Weißabgleich deaktivieren
    ctrl.AnalogueGain = 9.0      # Helligkeit durch analogen Gain (ohne
Rauschen) erhöhen
    ctrl.ExposureTime = 2300     # Belichtungszeit in Mikrosekunden (z. B.
2300 = 2.3ms)
    ctrl.NoiseReductionMode = 4 # Rauschreduzierung aktivieren
    ctrl.Sharpness = 6           # Bildschärfe einstellen

#time.sleep(2)
print("Starten der Bilderkennung")

while True:
    # Einlesen eines Frames von der Kamera
    im = picam2.capture_array()

    # Konvertieren des Bildes in den HSV-Farbraum
    hsv = cv2.cvtColor(im, cv2.COLOR_BGR2HSV)

    # Iterieren durch alle definierten Farben in colorProfiles
    for i in range(4):
        name, hsv_lower, hsv_upper = colorProfiles(i)

        mask = cv2.inRange(hsv, hsv_lower, hsv_upper)

        # Erkennen von Konturen in der Maske
        contours, hierarchy = cv2.findContours(mask.copy(),
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)

        if contours:
            # Sortieren der Konturen nach Flächengröße (absteigend)
            biggest = sorted(contours, key=cv2.contourArea,
reverse=True)[0]
            area = cv2.contourArea(biggest)

            # Überprüfen ob die Fläche der vorgegebenen Mindestgröße
entspricht
            if area >= min_area:
                x, y, w, h = cv2.boundingRect(biggest)

                # Einrahmen der erkannten Kontur
                cv2.rectangle(im, (x, y), (x + w, y + h), (0, 255, 255),
2)

                # Label mit Namen und Koordinaten im OpenCV
Vorschaufenster
                text = f"{name} ({x}, {y})"
                cv2.putText(im, text, (x, y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
```

```
                # Überprüfung der Flag für jede Farbe um sofortige
mehrfachzählung zu vermeiden (Alternativ Kombination mit time.sleep um
Wartezeit hinzuzufügen)
                if name == "Blau" and blueObject == False:
                    counter += 1
                    blueObject = True
                    print(f"Farbe: {name}, Counter: {counter}")
                elif name == "Rot" and redObject == False:
                    counter += 1
                    redObject = True
                    print(f"Farbe: {name}, Counter: {counter}")
                elif name == "Gelb" and yellowObject == False:
                    counter += 1
                    yellowObject = True
                    print(f"Farbe: {name}, Counter: {counter}")
                elif name == "Rot2" and redObject2 == False:
                    counter += 1
                    redObject2 = True
                    print(f"Farbe: {name}, Counter: {counter}")

            else:
                # Zurücksetzen der Flag bei zu kleiner Fläche
                if name == "Blau":
                    blueObject = False
                elif name == "Rot":
                    redObject = False
                elif name == "Gelb":
                    yellowObject = False
                elif name == "Rot2":
                    redObject2 = False
        else:
            # Zurücksetzen der Flag falls keine Kontur gefunden worden
ist
            if name == "Blau":
                blueObject = False
            elif name == "Rot":
                redObject = False
            elif name == "Gelb":
                yellowObject = False
            elif name == "Rot2":
                redObject2 = False

    # Anzeigen des erkannten Objekts
    cv2.putText(im, f"Anzahl erkannter Objekte: {counter}", (30, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
    cv2.imshow("HSV",hsv)
    cv2.imshow("Kamera Vorschau", im)

    # Beenden des Programms mit der Taste 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cv2.destroyAllWindows()
```

MAVSDK-Python - Bibliothek (https://github.com/mavlink/MAVSDK-Python):

3. Datenaustausch zwischen Raspberry Pi und Pixhawk

a. Abruf von Telemetrie-Parametern

```
#!/usr/bin/env python3

'''
Programm zum Auslesen von Telemetrie-Parametern
'''

import asyncio
from mavsdk import System

async def run():
    # Connect to the drone
    drone = System()
    #await drone.connect(system_address="udp://:14540")
    await drone.connect(system_address="serial:///dev/ttyAMA0:57600")

    # Get the list of parameters
    all_params = await drone.param.get_all_params()

    # # Iterate through all int parameters
    # for param in all_params.int_params:
    #     if param.name == 'TRIG_MODE':
    #         print(f"{param.name}: {param.value}")

    # Iterate through all int parameters
    for param in all_params.int_params:
        print(f"{param.name}: {param.value}")

    for param in all_params.float_params:
        print(f"{param.name}: {param.value}")

# Run the asyncio loop
asyncio.run(run())
```

b. Vorgabe von Wegpunkten (bzw. Positionen im Offboard-Modus)

```
#!/usr/bin/env python3

"""
Caveat when attempting to run the examples in non-gps environments:

`drone.offboard.stop()` will return a `COMMAND_DENIED` result because it
requires a mode switch to HOLD, something that is currently not supported
in a
non-gps environment.
"""

import asyncio

from mavsdk import System
from mavsdk.offboard import (OffboardError, PositionNedYaw)
```

```python
async def run():
    """ Does Offboard control using position NED coordinates. """

    drone = System()
    #await drone.connect(system_address="udp://:14540")
    await drone.connect(system_address="serial:///dev/ttyAMA0:57600")

    print("Waiting for drone to connect...")
    async for state in drone.core.connection_state():
        if state.is_connected:
            print(f"-- Connected to drone!")
            break

#     print("Waiting for drone to have a global position estimate...")
#     async for health in drone.telemetry.health():
#         if health.is_global_position_ok and health.is_home_position_ok:
#             print("-- Global position estimate OK")
#             break

    print("-- Arming")
    await drone.action.arm()

    print("-- Setting initial setpoint")
    await drone.offboard.set_position_ned(PositionNedYaw(0.0, 0.0, 0.0,
0.0))

    print("-- Starting offboard")
    try:
        await drone.offboard.start()
    except OffboardError as error:
        print(f"Starting offboard mode failed \
                with error code: {error._result.result}")
        print("-- Disarming")
        await drone.action.disarm()
        return

    print("-- Go 0m North, 0m East, -1m Down \
            within local coordinate system")
    await drone.offboard.set_position_ned(
            PositionNedYaw(0.0, 0.0, -1.0, 0.0))
    await asyncio.sleep(10)

    print("-- Go 1m North, 0m East, -1m Down \
            within local coordinate system, turn to face East")
    await drone.offboard.set_position_ned(
            PositionNedYaw(1.0, 0.0, -1.0, 0.0))
    await asyncio.sleep(10)

    print("-- Go 1m North, 1m East, -1m Down \
            within local coordinate system")
    await drone.offboard.set_position_ned(
            PositionNedYaw(1.0, 1.0, -1.0, 0.0))
    await asyncio.sleep(15)

    print("-- Go 0m North, 0m East, 0m Down \
            within local coordinate system, turn to face South")
    await drone.offboard.set_position_ned(
            PositionNedYaw(0.0, 0.0, 0.0, 0.0))
    await asyncio.sleep(10)

    print("-- Stopping offboard")
```

```
    try:
        await drone.offboard.stop()
    except OffboardError as error:
        print(f"Stopping offboard mode failed \
                with error code: {error._result.result}")


if __name__ == "__main__":
    # Run the asyncio loop
    asyncio.run(run())
```

c. Synchronisieren von Bildern mit Telemetrie-Daten und speichern in CSV-Datei

```
#!/usr/bin/python3

"""
Beschreibung: Programm zur Bildaufnahme mit synchronisierten Telemetrie-
Daten (z.B. Batteriestatus) und Abspeichern in CSV-Datei
Anpassbare Parameter:
- Anzahl der Bilder: Zeile 34
- Zeitintervall zwischen den Bildaufnahmen: Zeile 53
- Dateispeicherort: Zeile 18
"""

import asyncio
from mavsdk import System            # Bibliothek zur Kommunikation mit
der Drohne (Pixhawk)
from picamera2 import Picamera2      # Kamerasteuerung
from datetime import datetime        # Für Zeitstempel der Bilder
import os                            # Für Speicherverzeichnis

# Speicherverzeichnis erstellen (falls nicht vorhanden)
SAVE_PATH = '/home/aviator/Desktop/battery_images'
os.makedirs(SAVE_PATH, exist_ok=True)

# Kamera initialisieren
picam2 = Picamera2()

async def main():
    # Drohnenobjekt initialisieren und Verbindung herstellen
    drone = System()
    await drone.connect(system_address="serial:///dev/ttyAMA0:57600")
    print("Verbunden mit Pixhawk --> Abruf von Telemetriedaten")

    # Kamera starten
    picam2.start()

    # Bilder aufnehmen und dazugehörige Telemetriedaten speichern
    for i in range(3):  # Anzahl der Bilder (anpassbar)
        # Zeitstempel für fortlaufenden Bildnamen erzeugen
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        image_filename = f'image_{timestamp}.jpg'

        # Bild im gewählten Verzeichnis speichern
        picam2.capture_file(f'{SAVE_PATH}/{image_filename}')

        # Batteriestatus abrufen
        battery_data = await get_battery_status(drone)
```

```
        # Synchronisierte Telemetriedaten (Batteriestatus) in eine CSV-
Datei schreiben
        with open(f"{SAVE_PATH}/image_battery_log.csv", "a") as log_file:
            log_file.write(f"{image_filename},
{round(battery_data.voltage_v, 2)}V,
{battery_data.remaining_percent}%\n")

        print(f"Bild {image_filename} gespeichert mit Batteriestatus:
{round(battery_data.voltage_v, 2)}V, "
              f"{battery_data.remaining_percent}%.")

        # Zeitintervall zwischen Bildaufnahmen (anpassbar)
        await asyncio.sleep(3)  # 10 Sekunden Pause zwischen den
Aufnahmen

    # Kamera stoppen
    picam2.stop()

async def get_battery_status(drone):
    """
    Auslesen des aktuellen Batteriestatus (Spannung und verbleibende
Prozent)
    von der Drohne mithilfe der MAVSDK-Telemetrie.
    """
    async for battery in drone.telemetry.battery():
        return battery

# Hauptfunktion starten (asyncio Event Loop)
asyncio.run(main())
```